# SQL PRACTICE (GULSHAN)

Getting Started:-

CREATE DATABASE

SQLQuery1.sql - loc...QL (MSI\gulsh (68))*

```
CREATE DATABASE PracticeSQL;
USE PracticeSQL;
```

CREATE TABLE

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) UNIQUE NOT NULL
);
CREATE TABLE Employees (
    EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age >= 18),
    Salary INT,
    DepartmentID INT,
    JoinDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_EmpDept FOREIGN KEY (DepartmentID)
    REFERENCES Departments(DepartmentID)
```

INSERT INTO <*tablename*>

```
INSERT INTO Departments VALUES (1, 'HR'), (2, 'IT'), (3, 'Finance');
INSERT INTO Employees (Name, Age, Salary, DepartmentID)
VALUES
('Alice', 25, 60000, 1),
('Bob', 28, 50000, 2),
('Charlie', 30, 70000, 1),
('David', 24, 45000, 3),
('Eve', 27, 65000, 2);
```

SELECT, WHERE, ORDER BY

```sql
SELECT * FROM Employees;
SELECT * FROM Employees WHERE Salary > 55000;
SELECT * FROM Employees ORDER BY Salary DESC;
```

JOINS

INNER JOIN (matching only)

```sql
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

100 %

Results | Messages

| | Name | DepartmentName |
|---|---|---|
| 1 | Alice | HR |
| 2 | Bob | IT |
| 3 | Charlie | HR |
| 4 | David | Finance |
| 5 | Eve | IT |

LEFT JOIN (all from left)

```sql
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

100 %

Results | Messages

| | Name | DepartmentName |
|---|---|---|
| 1 | Alice | HR |
| 2 | Bob | IT |
| 3 | Charlie | HR |
| 4 | David | Finance |
| 5 | Eve | IT |

RIGHT JOIN(right table full)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

100 %

Results | Messages

| | Name | DepartmentName |
|---|---|---|
| 1 | Alice | HR |
| 2 | Charlie | HR |
| 3 | Bob | IT |
| 4 | Eve | IT |
| 5 | David | Finance |

GROUP BY (group rows)

```
SELECT DepartmentID, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DepartmentID;
```

100 %

Results | Messages

| | DepartmentID | AvgSalary |
|---|---|---|
| 1 | 1 | 65000 |
| 2 | 2 | 57500 |
| 3 | 3 | 45000 |

HAVING (filter groups)

```sql
SELECT DepartmentID, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DepartmentID
HAVING AVG(Salary) > 55000;
```

100 %

Results | Messages

| | DepartmentID | AvgSalary |
|---|---|---|
| 1 | 1 | 65000 |
| 2 | 2 | 57500 |

SUBQUERIES(IN WHERE)

```sql
SELECT Name
FROM Employees
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

100 %

Results | Messages

| | Name |
|---|---|
| 1 | Charlie |

Subquery with IN

```sql
SELECT Name
FROM Employees
WHERE DepartmentID IN (
    SELECT DepartmentID FROM Departments WHERE DepartmentName = 'IT
);
```

100 %

Results | Messages

| | Name |
|---|---|
| 1 | Bob |
| 2 | Eve |

CONSTRAINTS

PRIMARY KEY + CHECK + DEFAULT

```sql
CREATE TABLE TestTable (
    ID INT PRIMARY KEY,
    Age INT CHECK (Age >= 18),
    CreatedOn DATETIME DEFAULT GETDATE()
);
```

FOREIGN KEY

```sql
ALTER TABLE Employees
ADD CONSTRAINT FK_Dep
FOREIGN KEY (DepartmentID)
REFERENCES Departments(DepartmentID);
```

## VIEWS(Virtual Tables)

Create View

```sql
CREATE VIEW HighSalaryEmployees AS
SELECT Name, Salary
FROM Employees
WHERE Salary > 60000;
```

See View

```sql
SELECT * FROM HighSalaryEmployees;
```

100 %

Results | Messages

| | Name | Salary |
|---|---|---|
| 1 | Charlie | 70000 |
| 2 | Eve | 65000 |

## STORED PROCEDURES

```
CREATE PROCEDURE GetEmployeesByDept
@DeptID INT
AS
BEGIN
    SELECT * FROM Employees WHERE DepartmentID = @DeptID;
END;
```

Executing Procedures

```
EXEC GetEmployeesByDept 2;
```

100 %

Results | Messages

| | EmployeeID | Name | Age | Salary | DepartmentID | JoinDate |
|---|---|---|---|---|---|---|
| 1 | 2 | Bob | 28 | 50000 | 2 | 2026-02-06 11:42:14.400 |
| 2 | 5 | Eve | 27 | 65000 | 2 | 2026-02-06 11:42:14.400 |

## FUNCTIONS

Scalar Function

```
CREATE FUNCTION GetBonus (@Salary INT)
RETURNS INT
AS
BEGIN
    RETURN @Salary * 0.10;
END;
```

Use Function

```sql
SELECT Name, dbo.GetBonus(Salary) AS Bonus
FROM Employees;
```

100 %

Results | Messages

| | Name | Bonus |
|---|---|---|
| 1 | Alice | 6000 |
| 2 | Bob | 5000 |
| 3 | Charlie | 7000 |
| 4 | David | 4500 |
| 5 | Eve | 6500 |

TRANSACTIONS

```sql
BEGIN TRANSACTION;

UPDATE Employees SET Salary = Salary + 5000 WHERE Name = 'Bob';

ROLLBACK;   -- undo
-- COMMIT; -- save
```

INDEXES

```sql
CREATE NONCLUSTERED INDEX idx_salary
ON Employees (Salary);
```

table PracticeSQL.dbo.Employees

**CURSORS**

SQLQuery1.sql - loc...QL (MSI\gulsh (68))*

```sql
DECLARE @Name VARCHAR(50), @Salary INT;

DECLARE emp_cursor CURSOR FOR
SELECT Name, Salary FROM Employees;

OPEN emp_cursor;

FETCH NEXT FROM emp_cursor INTO @Name, @Salary;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @Name + ' earns ' + CAST(@Salary AS VARCHAR);
    FETCH NEXT FROM emp_cursor INTO @Name, @Salary;
END;

CLOSE emp_cursor;
DEALLOCATE emp_cursor;
```

100 %

Messages

```
Alice earns 60000
Bob earns 50000
Charlie earns 70000
David earns 45000
Eve earns 65000

Completion time: 2026-02-06T16:11:27.5010809+05:30
```